

Overview of the Prometheus Methodology

In the previous chapter, we covered the concepts that are used by intelligent software agents. However, there is still a gap – knowing these concepts does not answer the practical question ‘How do I build a software system based on intelligent agents?’

The next few chapters provide a detailed answer to this question. What is missing is a *process* that breaks down ‘build a software system’ into smaller steps that are followed in order to specify, design and build agent-oriented systems.

In addition to high-level steps such as ‘specify the system’ or even ‘identify the system’s goals’, a usable methodology needs to provide detailed guidelines explaining *how* these steps are carried out. Often, these guidelines are expressed as a collection of heuristics and examples: it is difficult to give hard rules in a general-purpose methodology, and the design decisions often concern trade-offs. Processes and heuristics can help designers identify the decision points and the reasons for making various choices, but cannot make choices for them.

As the process is followed, *design artifacts* are produced – these are used to capture information about the system and its design. For example, in UML, a class diagram captures the classes in a system and their relationships. Design artifacts vary in formality and size – from a weighty tome written in natural English, to diagrams on a white board, to fully formal specifications written in a precisely defined notation.

Although design artifacts could all be specified using a natural language such as English, this is undesirable for a number of reasons: natural languages are unstructured, inherently ambiguous and do not capture certain types of information well. Also, English cannot be easily supported by software tools. Finally, many software engineers do not feel comfortable writing.

As a result, design artifacts are often specified in some formal or, more usually, semi-formal *notation*. A notation can be graphical or textual (including free English text, structured text such as forms, etc.).

Thus, a methodology should provide a *process* with *detailed guidelines* (including examples and heuristics) and *notations* that are used to describe the design artifacts.

In the remainder of this chapter, we

- discuss why existing (non-agent) methodologies are not appropriate for designing agent systems;
- give a brief overview of the Prometheus methodology;
- provide guidelines regarding the use of Prometheus; and
- briefly relate Prometheus to existing agent-oriented methodologies.

3.1 WHY A NEW METHODOLOGY?

One question that might be asked is ‘Why do we need another new methodology’? Indeed, there are many existing methodologies for designing software. In particular, object-oriented analysis and design have been extensively studied and developed. Is it not possible to use object-oriented techniques to build agent systems?

The short answer is ‘Not well!’. Although agents and objects do have similarities, the differences are significant. It is possible to use object-oriented analysis and design techniques to design agent systems. However, the fit is not natural and the resulting design is less likely to make good use of agents.

For example, one important aspect of agents is that they are *proactive*, that is, that they pursue their own agenda over time. This is realized in terms of goals. A methodology that supports proactive agents needs to support the explicit modelling of goals, which is not generally a part of object-oriented methodologies. By contrast, Prometheus models goals and thus supports the design of proactive agents.

Some other areas where Prometheus differs significantly from object-oriented methodologies include the following:

- The provision of a process for determining the types¹ of agents in the system.
- Treating messages as components in their own right, not just as labels on arcs. This allows a message (or an event) to be handled by multiple plans, which is crucial to achieving flexibility and robustness.
- Distinguishing percepts and actions from messages, and looking explicitly at percept processing.

Agents are situated in an environment, and it is important to define the interface between agents and their environment. Percept processing is often important for agents that are situated in the real world and take their percepts from noisy devices such as video cameras.

¹Agents existing at run time are instances of agent types. For example, in designing an online book store, we may identify agent types such as a Sales Assistant agent type. At run time, there may be multiple agents that are instances of this type.

- Distinguishing passive components (data, beliefs) from active components (agents, capabilities, plans): with object-oriented modelling, everything is modelled as (passive) objects.
- One view of agents (the intentional stance (Dennett 1987)) ascribes *mental attitudes*, such as beliefs, and desires to agents. If we subscribe to this view, then we would like the design methodology to address these aspects. Existing non-agent methodologies do not ascribe mental attitudes to software components. It is worth pointing out that some agent-oriented methodologies (e.g. MaSE) do not subscribe to this view, and, consequently, do not address mental attitudes. Others, including Prometheus, do capture mental attitudes during the analysis and design processes.

Note that although there are clear differences between Prometheus and object-oriented methodologies, there are also commonalities. Although current object-oriented methodologies are not sufficient for engineering agent-oriented software, they *are* relevant – agents are software, and, indeed, many aspects of the Prometheus methodology have been based on object-oriented methods and notations. For example, use case scenarios are adapted from standard practice (Jacobson *et al.* 1992); interaction diagrams are UML sequence diagrams; AUMl (<http://www.auml.org/>) (itself an extension of UML) is used directly, and the Rational Unified Process (RUP) (Kruchten 1998) and Prometheus share a similar approach to applying an iterative process over clearly delineated phases.

We have argued that the current mainstream methodologies, as exemplified by UML and RUP, do not provide sufficient support for producing good agent-oriented designs. We now give a brief overview of the agent-oriented methodology Prometheus, which will be the focus of much of this book.

3.2 PROMETHEUS: A BRIEF OVERVIEW

The Prometheus² methodology defines a detailed process for specifying, designing, implementing and testing/debugging³ agent-oriented software systems. In addition to detailed processes (and many practical tips), it defines a range of artifacts that are produced along the way. Some of these artifacts are kept, and some are only used as ‘stepping stones’. Some of the artifacts are graphical while others are structured text (i.e. forms).

Prometheus’ artifacts relate back to the agent concepts that were introduced in the previous chapter. For example, actions and percepts are captured in the system specification phase; the detailed design phase results in plans, events and beliefs; and the entities used in the various overview diagrams correspond directly to the concepts.

Note that all of the artifacts are structured. This is important in order to be able to provide tool support for the methodology.

The *Prometheus* methodology consists of three phases, depicted in Figure 3.1.

²Prometheus was the wisest Titan. His name means ‘forethought’ and he was able to foretell the future. Prometheus is known as the protector and benefactor of man. He gave mankind a number of gifts including fire. (from <http://www.greekmythology.com/>)

³Testing and debugging are not covered in this book. Poutakidis *et al.* (2002, 2003) have proposed a debugging method and tool to debug agent interactions, but this is not yet mature enough to be usable for debugging real systems.

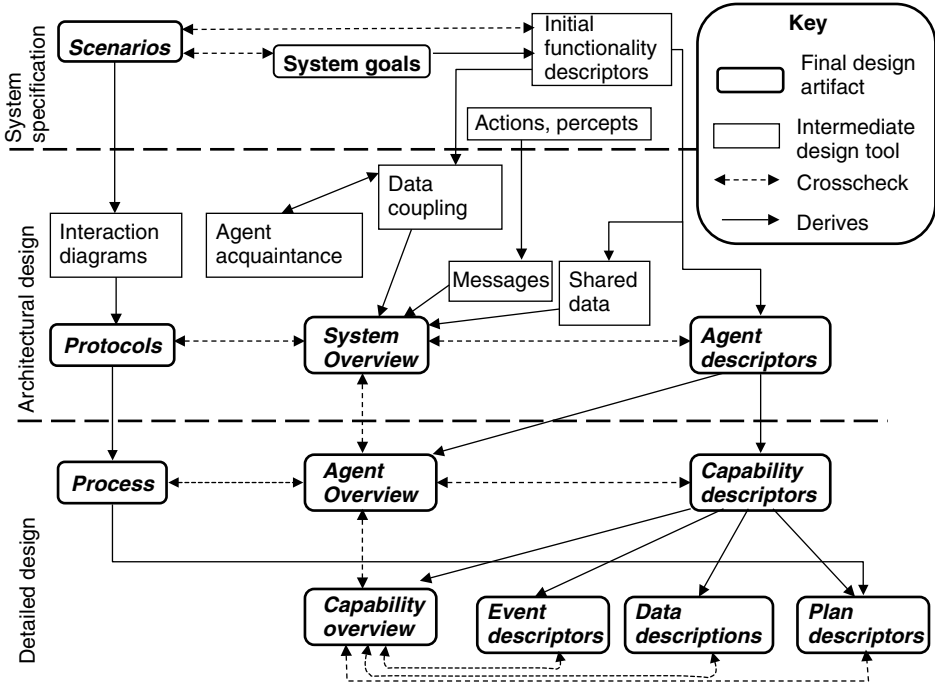


Figure 3.1 The phases of the Prometheus methodology

1. The *system specification phase* focuses on identifying the goals and basic functionalities of the system, along with inputs (percepts) and outputs (actions).
2. The *architectural design phase* uses the outputs from the previous phase to determine which agent types the system will contain and how they will interact.
3. The *detailed design phase* looks at the internals of each agent and how it will accomplish its tasks within the overall system.

A fourth phase is implementation, which is omitted from Figure 3.1 because its details depend on the implementation platform chosen.

The following description of these phases is intended to give a rough feel for the overall structure of the methodology, so that when reading the following chapters, where Prometheus is described in detail, you have some idea of how the details fit into the bigger picture.

3.2.1 SYSTEM SPECIFICATION

The system specification phase, described in detail in Chapter 4, focuses on the following:

- Identifying the **system goals**.
- Developing use case **scenarios** illustrating the system's operation.

- Identifying the basic **functionalities** of the system.
- Specifying the interface between the system and its environment in terms of **actions** and **percepts**.

These four steps do not proceed in sequence; rather, one shifts between them. For example, after adding a goal, we might add a use case scenario illustrating how this goal is achieved. The scenario might include new goals and/or functionalities.

System goals are useful in capturing, at a high level, what the system needs to be able to do. Because goals are a fairly high-level description, they tend to be less likely to change over time than functionalities. Identifying the goals results in a collection of goals, each with a name and description. We also capture the relationship between goals and sub-goals.

Use case scenarios are examples of the system's operation. They are useful in that they are more specific and concrete than goals and thus tend to be more accessible and easier to visualize. However, they do not tell the whole story. Roughly speaking, a use case scenario consists of a sequence of steps that occurs during the system's operation, along with a description of the context in which this sequence of steps could occur. Use case scenarios are borrowed from object-oriented design; however the details differ.

Functionalities are chunks of behaviour formed by grouping related goals, together with related data, percepts and actions. For example, in defining an electronic bookstore we may include functionalities such as "Delivery Handling Functionality" or "Stock Purchasing Functionality". These functionalities should be specific enough that they can be described adequately in a sentence or two.

We also specify the environment in which the agents will be situated. This is defined in terms of *actions* (ways that the agents affect the environment), and *percepts* (incoming information from the environment).

3.2.2 ARCHITECTURAL DESIGN

The architectural design phase, detailed in Chapters 5, 6 and 7, focuses on the following:

- Deciding what **agent types** will be implemented and developing the **agent descriptors**.
- Capturing the system's overall (static) structure using the **system overview diagram**.
- Describing the dynamic behaviour of the system using **interaction diagrams** and **interaction protocols**.

A major decision to be made during the architectural design is which agent types should exist. This is done by grouping functionalities into agent types. Each agent type consists of one or more functionalities. Given a set of functionalities, there is a large number of possible groupings. Deciding on a reasonable grouping is guided by considerations of coupling and cohesion. A **data coupling diagram** is used to help guide this

decision, and an **agent acquaintance diagram** is used to help assess whether a reasonable decision was made. Once a grouping is chosen, each agent type is described using an **agent descriptor** form.

Once we have decided upon the agents in the system, we identify which agents react to which percepts, as well as which agents perform particular actions on the external environment. In addition, we specify the **messages** between agents and determine the major **shared data** repositories. These items form the overall design of the system and are depicted in the **system overview diagram**. The system overview diagram is perhaps the single most important product of the design process. It ties together agents, data, external input and output, and shows the communication between agents.

The system overview diagram shows the pathways of communication – which agents talk to which other agents – but not the *timing* of communication – which messages are followed by which other messages. The timing of communication is captured initially in the **scenarios**: whenever an activity by an agent is followed by an activity of another agent, there is, implicitly, communication between them. This communication is depicted explicitly in **agent interaction diagrams**. Like scenarios, interaction diagrams depict one possible sequence of messages between agents. In order to describe all possible interactions, we develop **interaction protocols**, depicted using Agent UML (AUML⁴) (Huget *et al.* 2003).

3.2.3 DETAILED DESIGN

Detailed design focuses on developing the internal structure of each of the agents and how it will achieve its tasks within the system.

In essence, we progressively refine each agent by defining **capabilities** (modules within the agent), internal **events**, **plans** and detailed **data** structures. This process begins by describing agents' internals in terms of capabilities. The internal structure of each capability is then described, optionally using or introducing further capabilities⁵. These are refined in turn until all capabilities have been defined. At the bottom level, capabilities are defined in terms of plans, events and data. In parallel, we continue to work on developing the dynamics of the system by refining interaction protocols into **process specifications**.

The detailed design process presented in this book is split into two parts. In Chapter 8, we focus on the following:

- The refinement of agents in terms of capabilities, giving the **agent overview diagram** and **capability descriptors**; and
- The development of **process specifications**.

This part of the process neither assumes nor relies on any particular implementation platform or architecture.

⁴We use the revised version of AUML currently being developed. For a brief introduction, see Appendix C.

⁵Capabilities are allowed to be nested within other capabilities and thus this model allows for as many layers within the detailed design as are needed in order to achieve an understandable complexity at each level.

The agent overview and the capability descriptors are analogous to the system overview diagram and the agent descriptors, but now focussing within a single agent. The process specifications provide a detailed view of an individual agent's part in a particular process, as defined by the protocol specification at the global level. Each global protocol will have a number of corresponding local views, each of which defines the process from the point of view of a particular agent.

In Chapter 9, we continue the detailed design process, focussing on:

- Design of the plans within a capability and the events generated and handled by these plans, as captured in the **capability overview diagrams**.
- Specification of the algorithm within each plan, as well as associated data (or beliefs) and detailed specification of events. These are captured in **plan, data and event descriptors**.

The capability overview diagrams are similar in style to the agent overview and system overview diagrams, although plans must indicate which incoming event is the *trigger* event. Plans are part of the specification of the dynamics of the system and in developing them we take into account the process specifications, as well as structural information such as what triggers them. The descriptors provide the details necessary to move into implementation. Exactly what are the appropriate details for these descriptors will depend on aspects of the implementation platform.

It is only in this final part of the detailed design that we are committing to a particular implementation style. Specifically, we are assuming that plans are triggered by events and that it is possible to have multiple plans that handle a given event type, where the choice of plan to be used is determined at run time. This assumption corresponds to a whole class of implementation platforms (see Chapter 10) including BDI systems such as JACK and systems based on hierarchical task networks such as RETSINA.

3.3 GUIDELINES FOR USING PROMETHEUS

Before we discuss the details of Prometheus, we discuss a few issues involved in using Prometheus:

- Not following the methodology strictly
- The role of iteration and tool support
- When to use agents.

The Prometheus methodology is intended to be interpreted as a set of *guidelines*, not followed strictly. In developing Prometheus, we have had to steer a fine balance between committing to certain ways of doing things and keeping the methodology from becoming too complex and too specific. We hope that Prometheus, as is, will be useful for a large range of domains and users. However, we do not expect that Prometheus will be a perfect fit for all situations. For example, a small and simple agent system may not need to use capabilities; or an agent system may be simulating an existing (human) organization and thus the agent types may already be known, making the use of functionalities and of the data coupling diagram redundant.

In addition, Prometheus is still evolving. There are a number of types of systems that it does not yet handle well. For example, Prometheus as described in this book does not address agent teamwork or mobile agents.

The basic principle is that you should use your common sense and judgement. If you have less experience in the design of agent systems, you will probably want to follow all of the process steps described. However, as you become more experienced, you should use your judgement in applying the methodology.

ITERATIVE DEVELOPMENT

A key element of modern software engineering is *iteration*. Like any other complex human endeavour, such as writing a book, it is not possible to get everything right the first time. Instead of attempting to get things perfect before moving on (as in the waterfall model), modern software engineering processes such as RUP are iterative: although there are still clearly defined activities such as requirements specification, high-level design, detailed design, implementation and testing, these activities are not done in sequence a single time. Rather, the whole process is iterated and the emphasis gradually shifts. The first few iterations might involve primarily requirements-specification activities; however, later iterations will introduce other activities. Prometheus also adopts this approach. Although the following chapters describe the phases and activities in a sequential manner, we do not advocate that they be applied sequentially.

One issue that arises with any iterative process that modifies an existing design is that changes in one part of a design can introduce inconsistencies. This necessitates cross-checking as the design is developed and modified. Although it is possible to perform such cross-checking manually, this is tedious and, more importantly, error prone. Fortunately, it is possible to provide automated support that can detect certain inconsistencies. Our experience has been that this sort of automated cross-checking is very useful in an iterative process.

WHEN TO USE AGENTS

It is important to consider what parts of a system should be treated as agents and designed using an agent-oriented methodology (such as Prometheus), and also how the links between an agent-oriented sub-system and non-agent software can be designed and implemented.

Not all software components are best viewed, modelled and designed as agents. Sometimes, you will be designing a system where it makes sense to model it entirely as a multi-agent system. However, this is not always the case. Some sub-systems may not make sense or may not benefit from being viewed as a collection of agents. For example, an image-processing sub-system that extracts the position of a ball from video frames will not benefit from being viewed as an agent or as a system of agents.

How can we identify which parts of a system should be viewed as agents (and which parts should not)? The short, pragmatic answer is that we should use agents where they are more natural and offer a benefit. The following questions can be used to help identify components that should be treated as agents (if the answers are ‘yes’):

- Is it autonomous?
- Does it have goals?
- Viewed as an object, is it active (in the sense of having internal threads that run concurrently with the rest of the system)?
- Does it do multiple things at once? If so, does it need to reason about interaction between the different activities?
- Does it need to change the way it is doing things on the basis of changes in its environment?

If the answers to these questions are mostly ‘yes’, then you should probably think of the components as agents (and design them accordingly).

EXTERNAL DATA AND CODE

Assuming that some part of the system will be designed as a multi-agent system using Prometheus and some other parts will be designed using other methods, how can the designs be integrated? There are two approaches: we can either view a non-agent part as being an external sub-system with which the agent system as a whole interacts, or as being an ability of a particular agent type.

The system overview diagram used in Prometheus allows for external data and for external ‘library code’ to be specified. External data can be accessed by various agent types, and this is specified in the design of the agent system. External library code can also be used by agents. The precise mechanism varies: it can be viewed as communicating with the agent system via messages, or as delivering percepts and accepting action requests. The specification of the external sub-system, if it is being built (as opposed to simply using existing code), can be done using whatever methodology is appropriate: object-oriented design, database design, or some other established method.

Alternatively, we can view non-agent parts as being *within* a certain agent type. Agents can include data. This data can be specified, structured and designed in many ways. Some data might be best viewed as a relational table, other data might be best viewed as objects. In very simple cases, data might just be provided data types (e.g. an agent’s velocity could just be a floating point number). In addition to including data, agents can also include any existing code by including the appropriate function call within an agent plan. This is viewed as an “internal action”.

3.4 AGENT-ORIENTED METHODOLOGIES

A large number of agent-oriented methodologies have been proposed in recent years (Brazier *et al.* 1997; Bresciani *et al.* 2002; Burmeister 1996; Burrafato and Cossentino 2002; Bush *et al.* 2001; Caire *et al.* 2001; Collinot *et al.* 1996; Cossentino and Potts 2002; Debenham and Henderson-Sellers 2002; DeLoach *et al.* 2001; Drogoul and Zucker 1998; Elammari and Lalonde 1999; Giunchiglia *et al.* 2002; Glaser 1996; Iglesias *et al.*

1999, 1997; Kendall *et al.* 1995; Kinny and Georgeff 1996; Kinny *et al.* 1996; Lind 2000; Odell *et al.* 2000; Shehory and Sturm 2001; Varga *et al.* 1994; Wagner 2002, 2003; Wooldridge *et al.* 2000). The aim of this book is to describe Prometheus, not to survey the many existing methodologies. Thus, this section is intentionally brief and incomplete. We encourage the interested reader to read the publications describing the various methodologies and the comparisons between methodologies that are beginning to appear (Cernuzzi and Rossi 2002; Dam 2003; Dam and Winikoff 2003; O'Malley and DeLoach 2001; Shehory and Sturm 2001; Sturm and Shehory 2002, 2003).

PROMETHEUS DESIGN GOALS

Prometheus is intended to be useful and usable by industry developers. In order to be usable, it must be described in sufficient detail and be complete, that is, cover all the necessary activities and phases. In order to be useful, it must support the design of realistically sized systems and thus must support an iterative mode of application (and, in particular, cannot assume a 'waterfall' process model). A consequence of this is that tool support is, in our opinion, highly desirable. We also believe that in order to be useful, Prometheus must focus on designing agents that are flexible and robust. In particular, the use of goals and plans as an implementation technology allows agents to be flexible and robust.

Thus, the design of Prometheus was guided by the following design criteria, aimed at ensuring that the methodology be useful and usable:

- Prometheus must be detailed enough to use and must be complete.
- Prometheus must support the development of agents based on goals, plans and beliefs.
- Prometheus must scale to large systems, and must not use a waterfall process model.
- Prometheus must facilitate tool support, and such tool support should be implemented.

Additionally, in order to ensure that Prometheus is actually useful to, and usable by, industry developers, it must be used by industry developers! In our work, we have both worked closely with the company Agent Oriented Software and have also taught Prometheus to undergraduate students.

These criteria set Prometheus apart from existing methodologies. Many of the existing agent-oriented methodologies are not yet ready to be used by industry developers: they are still under research, and either focus on specific aspects of agent design such as teamwork (i.e. do not provide a complete methodology), or are not described in sufficient detail. For example, many of the methodologies are only described in a small number of short (8 to 15 page) conference papers. Additionally, many methodologies do not provide tool support.

SOME SPECIFIC METHODOLOGIES

Of the long list of agent-oriented methodologies, there are some that *are* described in detail, that do offer tool support and that do appear to be ready for use. In particular, the

MaSE and Tropos methodologies are both complete, have been developed over a period of time (i.e. are mature) and both provide detailed descriptions.

The **Gaia** methodology (Wooldridge *et al.* 2000), like Prometheus, has been developed over a number of years by people experienced in building agent systems. However, we have found that the lack of a detailed design process – intentionally absent because of a desire for generality – meant that it did not provide sufficient support for the needs of those we were working with. There are similarities between Prometheus and Gaia for specification and architectural design. Our agent acquaintance diagrams are essentially the same as those used by Gaia, and the roles of Gaia are similar in concept to functionalities in Prometheus, although there are slightly different things that are considered.

The **Tropos** methodology (Bresciani *et al.* 2002; Giunchiglia *et al.* 2002) covers early requirements to detailed design. Its detailed design is oriented very specifically towards JACK as an implementation platform. Compared to Prometheus, Tropos provides an early requirements phase, which Prometheus does not (although it would certainly be possible to adapt Tropos' early requirements phase for use in Prometheus). Prometheus provides a more detailed process – particularly in the architectural design phase. Prometheus also provides tool support and cross-checking; tool support for Tropos is currently only in the form of a diagram editor rather than the consistency checking and automatic generation of some parts of the design that is part of the Prometheus Design Tool.

The **MaSE** methodology (DeLoach *et al.* 2001) is one of the few methodologies that appears to have significant tool support. However, MaSE is unsuitable for our purposes since it views agents ‘... merely as a convenient abstraction, which may or may not possess intelligence’ (DeLoach *et al.* 2001, p. 232). Thus, MaSE (intentionally) does not support the construction of plan-based agents that are able to provide a flexible mix of reactive and proactive behaviour. Rather, MaSE aims to be general and treats agents as ‘simple software processes that interact with each other to meet an overall system goal’ (DeLoach 2001, p. 232).

PASSI (Burrafato and Cossentino 2002; Cossentino and Potts 2002) is a recent addition to the list of methodologies. Although it has not, to the best of our knowledge, been described in detail, it appears to be complete, and provides tool support.

COMPARISONS

Comparisons of the existing methodologies are limited, but are beginning to appear. MaSE, Prometheus and Tropos are compared using a feature-based approach in which the assessment of each methodology against the criteria is validated using a survey of the developers of the methodology (and of students) (Dam and Winikoff 2003). This work is extended to include MESSAGE and GAIA and to also provide a comparative analysis of the models and processes of each of the methodologies (Dam 2003). Other comparisons between agent-oriented methodologies include (Shehory and Sturm 2001), (Cernuzzi and Rossi 2002), (Sturm and Shehory 2003).